

Prof. Etienne Messerli

SysLog2 / CSN

Labo ALU : Unité de calcul ALU "Arithmetic Logic Unit"

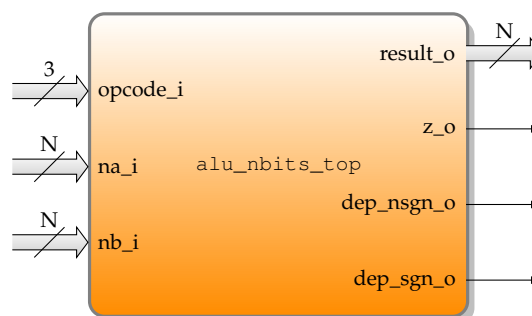
Contexte

Le but de ce projet est de concevoir, réaliser, décrire et tester une unité de calcul pour des opérations arithmétiques et logiques. Un tel circuit est communément nommé ALU pour Arithmetic Logic Unit. Ce circuit sera de type combinatoire.

Celui-ci correspond typiquement à l'unité de calcul présent dans un processeur. Dans ce cas, il y a plus d'opérations et un registre est présent à la sortie de l'ALU voire une banque de registres.

Composant à réaliser

Voici le symbole et les entrées/sorties de l'unité de calcul à réaliser :



Description des entrées/ sorties de l'unité de calcul :

Port	Direction	Taille	Description
opcode_i	Entrée	3	Entrée indiquant quelle opération le circuit doit exécuter. Il y a 8 opérations possibles.
na_i*	Entrée	N	Opérande. Il s'agit soit d'un nombre non signé ou signé en complément à deux (C2).
nb_i*	Entrée	N	Opérande utilisée dans certaines opérations. Il s'agit soit d'un nombre non signé ou signé en complément à deux (C2).
result_o*	Sortie	N	Résultat de l'opération effectuée (N bits).
z_o	Sortie	1	Indication d'un résultat nul (result_o égal à zéro).
dep_nsgn_o	Sortie	1	Dépassement de capacité pour nombres non signés.
dep_sgn_o	Sortie	1	Dépassement de capacité pour nombres signés.

TABLE 1 – Description des ports du module

* L'unité doit permettre de traiter des nombres non-signés ou signés en C2. C'est l'utilisateur qui doit traiter le dépassement correspondant au type de nombres utilisés lors d'une opération arithmétique.

La taille de l'unité ALU sera modifiable. Celle-ci sera définie par un générique (N) dans l'entité.

Spécification du fonctionnement souhaité

L'unité combinatoire à réaliser devra répondre aux spécifications données ci-après :

L'unité ALU est capable d'effectuer 6 opérations arithmétiques et 2 opérations logiques, pour des nombres signés et non signés sur N bits.

Le tableau ci-dessous vous donne la correspondance entre les valeurs de l'opcode_i et l'opération à réaliser :

opcode_i	Opération	Valeur sur result_o
000	Addition	$na_i + nb_i$
001	Multiplication	$2 \cdot na_i$
010	ET logique	$na_i \text{ and } nb_i$
011	Incrémentation	$na_i + 1$
100	Soustraction	$na_i - nb_i$
101	Soustraction	$nb_i - na_i$
110	OU logique	$na_i \text{ or } nb_i$
111	Décrémentation	$na_i - 1$

TABLE 2 – Table des opérations de l'ALU

Question : Expliquer comment vous pouvez obtenir à l'aide de votre ALU, le résultat des tests $na_i < nb_i$ ou $na_i > nb_i$. **Réponse à fournir dans le rapport.**

Contraintes

Vous devez optimiser la quantité de logique utilisée par votre solution. Ce critère doit guider vos choix lors de la décomposition du circuit. Nous avons vu que l'opération $A - B$ peut-être aussi réalisée par $A + (-B)$ soit $A + C2(B)$. Vous devez exploiter au mieux cette possibilité pour établir une architecture optimisée. Dans cette optique, le choix des codes, pour `opcode_i`, ont été choisis afin de permettre des optimisations.

Un bonus sera attribué au groupe qui utilisera le moins de LEs (Logic Element) du circuit CPLD Max-V.

Documents à rendre

Pour ce laboratoire, vous travaillerez par groupe de 2.

Vous devrez rendre à l'issue de ce travail de laboratoire 2 fichiers, soit :

- **Un fichier PDF** avec votre rapport de laboratoire comprenant l'ensemble de vos explications, étapes de conception, schémas (info : scanner schémas à la main), les preuves des vérifications réalisées, réponses aux différentes étapes et questions de- mandées, ainsi que les descriptions VHDL synthétisables
- **Une archive zip ou tar.gz** avec vos répertoires `src/...` et `src_tb/...` ainsi que vos scripts tcl

Vous devez déposer votre archive sur la page **26_HEIG-VD_SysLog2 / 26_HEIG-VD_CSN** du site Cyberlearn de la HES-SO (<https://cyberlearn.hes-so.ch/>)

Marche à suivre

1. Etablir une liste des différentes possibilités pour réaliser chacune des opérations demandées. Dans cette étape, il s'agit d'identifier les différentes ressources nécessaires pour chaque opération. Cela doit vous permettre de choisir une décomposition permettant d'optimiser les ressources matérielles.
 Etablir le schéma d'une ou plusieurs architectures de décomposition structurelle de l'ALU N-bits selon les conclusions obtenues précédemment. Puis choisir une solution.
2. Donner le schéma bloc de votre solution retenue pour l'ALU N bits avec les 8 opérations spécifiées. Chaque bloc devra être spécifié par un symbole CEI, une TDV ou une équation simple.
3. Décrire en VHDL synthétisable votre solution de l'ALU N bits établi au point précédent. Vous complétez le fichier `alu_nbits_top.vhd`. Vous décrierez dans un fichier séparé l'opération d'addition. L'entité fournie dans le fichier `alu_nbits_top.vhd` comprend déjà la déclaration d'un générique.
Conseil : réutiliser la solution de l'additionneur du labo précédent avec, si nécessaire, des adaptations.
4. Compilation des fichiers dans QuestaSim : Compléter le script tcl indiqué ci-dessous avec tous vos fichiers de votre solution. Le fichier contient la compilation du top (`alu_nbits_top.vhd`) et des fichiers de simulation.
Fichier à compléter : `comp_alu.tcl`
5. Vérification de l'unité de calcul `alu_nbits_top.vhd` à l'aide d'une simulation automatique dans Questasim. Vous effectuerez la simulation automatique pour N= 6bits Utilisez le script automatique : `run_alu_top_tb.tcl`
Preuve de la simulation : Copie fenêtre log du simulateur (avec message de loading)
6. Si nécessaire, vous disposez d'une simulation manuelle avec `console_sim.vhd` et la console REDS pour la recherche de pannes avec des cas particuliers.
 Utilisez le script automatique : `run_alu_top_sim.tcl`
7. Réaliser la synthèse du composant `alu_nbits_top.vhd` avec Quartus Prime, pour les trois tailles suivantes : N= 4, 8 ou 12. Vous indiquerez pour chaque cas le nombre de LEs (logique élément) obtenus. Analysez l'évolution de la quantité de logique.
8. Réaliser l'intégration de votre composant ALU N, avec N=8 bits, pour la carte Max-V 25-80 pôles. Vous devez utiliser le fichier `maxv_top.vhd`. Puis programmer le circuit **Max-V 5M570ZF256C5** de la carte.
 Reprendre les étapes pratiquées lors du labo *Intro_Bin_Lin*.
9. Tester votre circuit avec la carte ConsoleUSB2 et la console `REDS_console.tcl`
 Lancement de la console depuis un terminal : `sudo wish REDS_console.tcl`
 Vérification : vous devez établir une liste de cas pertinents pour valider votre circuit. Ensuite vous devez appliquer ceux-ci à votre montage
 Les informations sur la connexion des signaux de `alu_nbits_top.vhd` dans la Console_REDS sont à rechercher dans le fichier `maxv_top.vhd`
10. Vous devez faire valider le fonctionnement de votre solution par le professeur ou l'ingénieur.
11. **Question :** Expliquer comment vous pouvez obtenir à l'aide de votre ALU, le résultat des tests `na_i < nb_i` ou `na_i > nb_i`.

BON TRAVAIL!